

Agile Model-Driven Development in Practice

Yuefeng Zhang, UniqueSoft

Shailesh Patel, Motorola

// Combining model-driven development practices with agile techniques can significantly reduce software development cycle time and increase productivity and quality. //



BOTH THE AGILE development processes (such as scrum¹ and extreme programming²) and model-driven development (MDD) practices^{3,4} have been widely adopted in industry for years. To inherit the benefits of both, increasing amounts of results on introducing agility into MDD are being published in academia.^{5,6} For example, in *The Object Primer*, Scott Ambler describes how to combine extreme programming and MDD into agile

modeling.⁵ In previous work, Yuefeng Zhang presents a method of applying extreme programming practices to MDD for better productivity and quality.⁶

This article presents a different agile MDD process and its implementation for the development of a real-time telecommunication system. In particular, we show how to combine a system-level agile process (SLAP) and an MDD process into one to speed up de-

velopment rate, improve product quality, and shorten delivery cycle time.

SLAP and MDD Processes Overview

SLAP is an agile process adopted at Motorola that uses scrum¹ as a baseline and includes certain extreme programming practices.² As Figure 1 and Table 1 show, SLAP divides a software development life cycle into multiple iterations. Each iteration consists of three sprints: application requirements and architecture, development, and system integration feature testing (SIFT).

Each sprint is five weeks long and has the same set of activities (called its calendar). For example, the calendar for the development sprint is as follows: week 1 for sprint kickoff, application requirements analysis, and high-level design; week 2 for detailed design; week 3 for implementation; week 4 for formal technical review and rework; and week 5 for integration readiness and sprint postmortem meeting.

Starting at Sprint 3 in Iteration 2, all three sprints in different iterations go in parallel. For a given iteration, the requirements and architecture sprint aims to specify the application requirements and lay out the application architecture for the next iteration; the development sprint aims to implement the requirements and architecture from the previous iteration; and the SIFT sprint aims to test the implementation from the previous iteration.

Motorola's MDD process is based on the unified software development process.³ It divides a software development life cycle into multiple milestone phases over time (such as inception or elaboration), and each milestone phase consists of one or more iterations. Each iteration contains the same set of core development activities (such as require-

TABLE 1

Iterations and sprints in a system-level agile process (SLAP).

	Sprint 1	Sprint 2		Sprint n
Iteration 1	Requirements and architecture	Development	System integration feature testing (SIFT)	
Iteration 2		Requirements and architecture	Development	SIFT
.....			Requirements and architecture	Development
Iteration n				Requirements and architecture

TABLE 2

Mapping between SLAP and MDD in agile MDD.

SLAP	MDD
Sprints	Iterations
Requirements and architecture	Application requirements specification and architecture
Development	Requirements analysis and high-level design
	Detailed design
	Code generation and UML unit and integration testing
SIFT	Subsystem testing and system testing

ments analysis or high-level design) and follows a tailored V-Model process.

Agile MDD

The key in achieving agile MDD is to combine an agile process with an MDD process in a way that can inherit the benefits of both and at the same time avoid their shortcomings.

Process Overview

Our strategy of combining SLAP and MDD into agile MDD is to take SLAP as the backbone process and map MDD iterations and activities to the corresponding SLAP sprints and activities (see Table 2 and Figure 1).

In this way, like SLAP, an iteration in agile MDD still consists of three sprints. The sprint for requirements specification and architecture is the same as the one in SLAP. The development sprint includes requirements analysis, high-level design, detailed design, code gen-

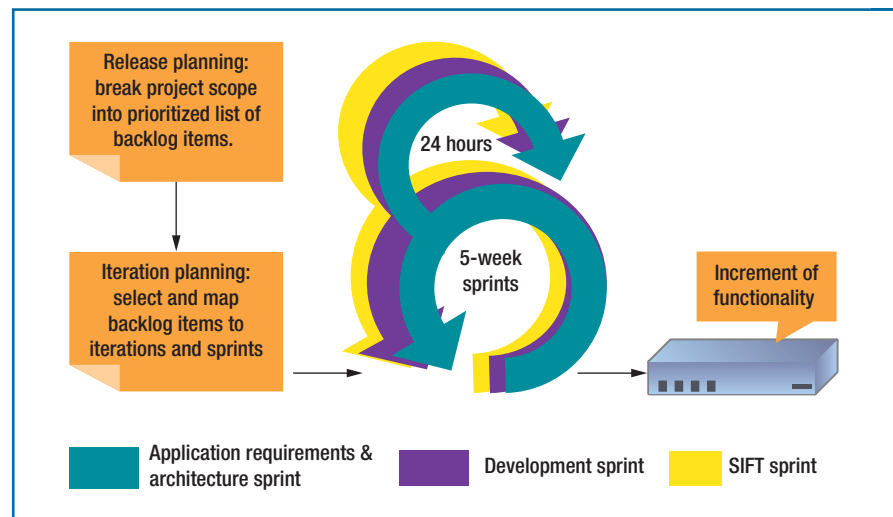


FIGURE 1. Agile model-driven development overview. The colors teal, purple, and yellow represent application requirements and architecture, development, and SIFT, respectively.

eration, UML unit testing, and integration testing. The SIFT sprint includes subsystem testing and system testing.

Implementation and Lessons Learned

The key in implementing agile MDD is

to figure out where and how to apply what agile practices in MDD.

Table 3 answers the where and what questions. In particular, it shows which MDD practices (*where*) correspond or relate to which agile practices

TABLE 3

MDD practices and corresponding (c) or related (r) agile practices.

		Agile practices							Agile management practices			
		Paired development	Test-driven development	Iterative and incremental development	Working software over comprehensive documentation	Automated regression testing	Continuous integration	Rapid feedback	Daily standup meeting	Daily development status tracking online	Short sprint cycle (5-week calendar)	Sprint postmortem meeting (retrospectives)
MDD practices	Modeling as coding	r										
	Paired modeling	c										
	Test-driven modeling		c									
	Iterative and incremental modeling			c								
	Modeling as live design documentation				r							
	Automated batch mode simulation					c						
	Continuous modeling						c					
	MDD tools chain							r				
Agile MDD management practices	Daily MDD plan update								r			
	Daily MDD task status update									r		
	MDD plan for the current sprint										r	
	MDD lessons learned in the past sprint (what to keep, change, try)											r

(*what*) in agile MDD. Here, the *corresponding* relationship represents that a given MDD practice corresponds to the counterpart of an agile practice, while the *relate* relationship means that the agile practice applies to the related MDD practice or that the MDD practice supports or enables the related agile practice.

In the rest of this subsection, we answer the how question—that is, how to apply agile practices to MDD and use MDD to support and enable agile practices. In addition, the following discussion also includes lessons learned in practice.

Modeling as coding. Here, we use UML as a high-level visual program-

ming language, and we use UML code generator as a UML compiler. Development is one-way: forward engineering from UML to C or C++. We apply paired development to UML modeling (that is, paired modeling).

We observed that it's essential for MDD tools to be able to trace code generation issues at the UML action language (SDL⁷ in our case) level back to the UML model level. MDD tools sometimes fail to do so.

Paired modeling. This corresponds to the paired development agile practice. Typically, two developers (perhaps a developer and a customer representative) work together on UML modeling for one sprint and rotate pairs per

sprint. A senior developer pairs with a junior developer, if possible.

This generally works in detecting and resolving modeling issues instantly, but not enough paring could be done all the time for various program priority reasons. Another difficulty is that rotating pairs asks for developers to change domain knowledge frequently. It's a challenge to minimize productivity impact due to ramp-up on domain knowledge. This issue is alleviated by discussing potential domain knowledge issues with developers prior to pair assignment so that domain knowledge transfer can be done in a controlled manner.

In addition to paired modeling, we also do paired review and training to

help individuals come up to speed on MDD tools and domain knowledge.

Test-driven modeling. This corresponds to the test-driven development agile practice. In this practice, we create UML sequence diagrams first and then use them to drive UML design and development of test cases.

At the beginning of a sprint, the focus of modeling and test case creation is on signal names and order. The data associated with a signal is ignored. Accordingly, the focus of UML state machines at the beginning of a sprint is on receiving and sending signals in the right order. The details of signal processing logic are ignored. Once all the test cases pass unit testing, we then add signal data fields into test cases and the details of signal processing logic into the state machines in the UML model.

This practice is most helpful for keeping a large number of model changes under control and preventing defects rather than finding defects after they occur in the UML model.

One difficulty is that we can't afford to represent all possible error scenarios in sequence diagrams. Another difficulty is that we can't use sequence diagrams to represent nonfunctional requirements. So, we created a manual test plan document to describe error scenarios and nonfunctional test cases.

Iterative and incremental modeling. This corresponds to the counterpart of agile practice. In this practice, we do UML modeling over multiple sprints, repeat the same set of development activities in each sprint, and create and evolve an executable model for each sprint with an increment of functionality on top of the model from the previous sprint.

The key is to keep the model executable in terms of both simulation and target platform testing. One major en-

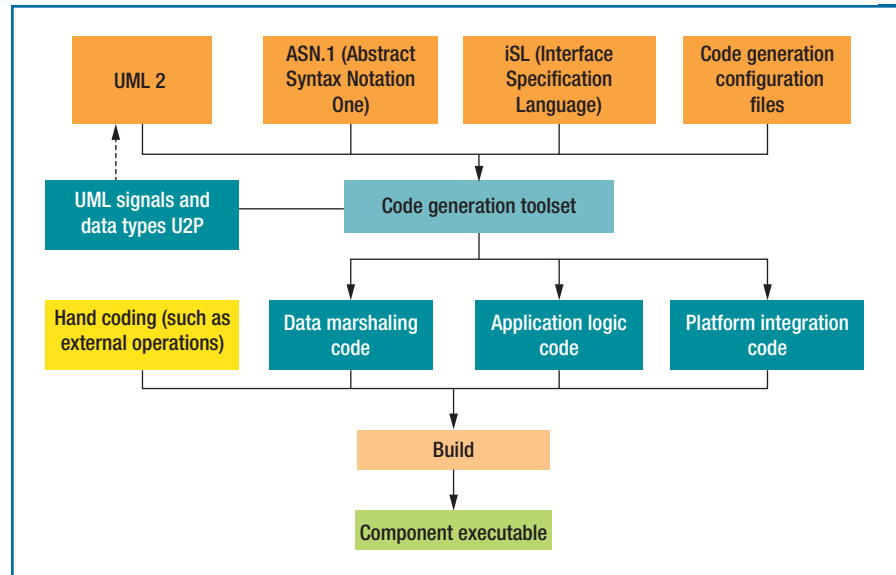


FIGURE 2. Target platform code generation workflow. The toolset generates application logic code from a UML model, generates data marshaling code from iSL and ASN.1 files, and generates platform integration code from configuration files.

abling factor is to maximize automation by an MDD tools chain, which includes maximized automatic code generation (see Figure 2) and automated regression testing by simulation.

Modeling as live design documentation. This is the result of applying the agile manifesto “working software over comprehensive documentation” to UML modeling. In this practice, we use the UML model as a live design document to minimize manual documentation.

Automated batch mode simulation. This corresponds to the agile practice of automated regression testing. In this practice, we develop and verify individual test cases first, add them to the list for automated batch mode execution, execute all the test cases in batch mode overnight, and finally analyze test results to find the root cause of the failed test cases.

We observed that the UML sequence diagram tracing capability of UML simulation tools is key for verifying and debugging new test cases quickly. To capture issues, we use a different dedicated developer each week to ana-

lyze testing results. In addition, MDD tools should provide a command-based interface for simulation builds and execution to minimize human interaction.

This practice plays a key role in automatically detecting UML modeling problems so that we can stop the modeling activity promptly to find the root cause of problems.

Continuous modeling. This corresponds to the agile practice of continuous integration. In this practice, we

- merge UML design frequently,
- perform automatic code generation for simulation and automated batch mode simulation many times in a sprint, and
- perform automatic code generation for target platform testing and SIFT two or more times in a sprint.

Figure 2 shows the target platform code generation workflow.

The automatic code generation is maximized in that it includes

- UML code generation,
- data marshaling code generation, and

- platform integration code generation.

To succeed in this practice, we must be able to merge UML model changes frequently. Unfortunately, the current UML modeling tool has some major graphical merging problems when developers merge many changes simultaneously. To alleviate this issue, we set up an integration branch and coordinate among developers so that we can merge a little at a time but frequently.

Automatic code generation and simulation are two rewarding methods for achieving mistake-proof development. They're also the fundamental enabler for UML model refactoring.

UML integration testing can help detect runtime issues with the UML model, data marshaling code, platform integration code, and other handwritten code such as external operations.

As part of SIFT, testing helps identify interoperability issues among different components within the application subsystem and the external interfaces between the subsystem and its environment.

MDD tools chain. In this practice, we value the MDD tools chain over individual tools, using it in particular to maximize automation. This enables rapid feedback and other agile practices.

Specifically, we use the following MDD tools as a tools chain:

- IBM Tau for UML modeling,
- IBM Tester for unit and integration test harness,
- a UML-to-test case data types conversion tool, and
- a code generation toolset for UML, data marshaling, and platform integration code generation.

As an example of the tools chain's usage, the code generation toolset can take a UML model from IBM Tau as

input and generate C/C++ code.

In choosing MDD tools, it's very important to consider not only the capability of individual tools, but also of interfacing to other tools to form a tools chain. To be productive, MDD tools should provide any necessary UML profile extensions and UML action language adjustments. In addition, UML language and tools training at the beginning of a project is important, but it's not enough. It's essential to provide agile MDD support upon request.

Agile MDD management. We use the agile project planning and management tool VersionOne (www.versionone.com) and the following agile MDD management practices for the development sprint management: a daily MDD plan update in daily standup meetings, a daily MDD task status update, frequent reviews of current sprint plans, and a sprint postmortem meeting for lessons learned.

Comparison with Stand-alone Agile and MDD Processes

From a high-level process view, agile MDD seems to be nothing more than a trivial summation of stand-alone agile and MDD processes. However, in reality, it's challenging to combine agile and MDD into a coherent process that can inherit the advantages of both while also avoiding their shortcomings. The major differences between agile MDD and stand-alone processes lie in streamlining MDD activities to align with agile processes.

Iterative and Incremental Development

The agile process applies iterations to all phases, from system requirements specification all the way down to system testing. This is achieved by tight collaboration of different teams through a feature team.

Motorola's stand-alone MDD process typically applies iterations from

the high-level design phase to the system integration testing phase only. All other phases follow a tailored V-Model process. This is justified because different teams collaborate loosely.

In agile MDD, to align with the agile process, MDD has been changed to enable end-to-end iterations—that is, apply iterations to all phases, from system requirements specification all the way down to system testing.

System Engineering

In both Motorola's agile and MDD processes, the typical system engineering artifacts are handwritten documents (such as FrameMaker files). Developers usually use UML diagrams only occasionally for documentation. No downstream reuse is attempted.

In agile MDD, to support short iteration cycle time, we streamline system engineering and development phases as follows. We create a system engineering UML model to precisely define a system architecture and its dynamic behavior in a layered fashion. In this way, a given application system is decomposed layer by layer until each leaf active class can be implemented as a separate OS process (which is handled as a separate UML model in the development sprint). We syntax check the system engineering model and create it with downstream reuse in mind, such as by reusing the interface, signals, and corresponding sequence diagrams of each leaf active class in the development sprint. Handwritten documents are created only for information that is not suitable for UML modeling, such as textural system requirements.

Coding

In the agile process, coding means to write programs in any chosen programming language. In MDD, the meaning of coding varies depending on the UML modeling tools in use. For a tool that only supports a state-oriented state machine and uses a programming lan-

guage as a UML action language, the tool typically generates a code skeleton. In this case, coding is a mixture of UML modeling and normal programming within the tool. If the tool supports reverse engineering, the coding can also include modifying the tool-generated source code files in the specifically marked regions and then reverse engineering the changes back into the model.

In agile MDD, coding is modeling, and the UML code generator is the UML compiler. In this manner, coding is one-way: forward engineering from UML to C/C++. This is achieved by using an MDD tools chain that supports UML transition-oriented state machines and can automatically generate fully executable code from a UML model.

Unit Testing

In the agile process, we use a unit testing tool (such as CppUnit for C++) to instantiate objects of classes under testing and call the methods of objects. We test each class as a black box. In MDD, unit testing is message driven. The way of performing unit testing varies, depending on the capability of tools. For example, some UML modeling tools support the execution of UML sequence diagrams. In this case, unit testing means to execute sequence diagrams. This type of testing is white box in nature and can apply to an individual active class, any active container class, or the entire model as a whole. A major issue with this method is that it can't test external operations. A common solution is to maintain a separate version of external operations for simulation only. Another difficulty is that, because the message data can be very complicated, it's difficult and error-prone to define and associate message data with sequence diagrams by hand. In addition, this testing method can only test the code generated for simulation.

In agile MDD, to streamline UML unit testing and integration testing, we use the MDD tools chain for unit testing. On the UML side, we use IBM Tau (a UML model verifier), and on the unit test harness side, we use IBM Tester. We use a co-simulation tool to bridge the communication between these two tools. In this manner, unit testing means to execute test cases through a test harness. Similar to MDD, this type of testing can apply to an individual active class, any active container class, or the entire model as a whole. A major difference is that this method is a combination of both black- and white-box testing: the test harness treats the UML model as a black box, and the UML model verifier treats the UML model as a white box in that it shows internal message exchanges among different active classes in the model. A major advantage is that this method can test not only normal UML elements but also external operations. Another advantage is that it can be adapted to perform integration and system testing.

Integration and System Testing

In the agile process, integration and system testing includes testing all the classes of an application system (or subsystem), related platform integration code, and data marshaling code as a whole. Testers use different test cases and procedures for integration and system testing.

In MDD, integration testing includes testing an entire UML model, related platform integration code, and data marshaling code as a whole. Both platform integration code and data marshaling code are typically created by hand. The test cases, procedures, and environments for MDD integration testing are different from those for integration testing in an agile process.

In agile MDD, to streamline unit, integration, and system testing, the UML unit testing environment is adapted to

implement both integration and system testing environments. Like MDD, the integration testing includes the entire UML model, related platform integration code, and data marshaling code as a whole. One difference is that in agile MDD, the tools chain automatically generates both platform integration code and data marshaling code. Another difference is that, because we test the UML model as a whole in unit testing, we reuse the same unit test cases for integration testing. In addition, the same tools chain automates both integration and system testing.

Test-Driven Development

In the agile process, test-driven development means to write unit test cases (that is, testing code with embedded class method calls) first, and then implement and test the class methods until the unit testing passes. In MDD, test-driven development means to create UML sequence diagrams first, then create the UML model according to the sequence diagrams, and, finally, modify and execute sequence diagrams until the UML unit testing passes.

In agile MDD, this practice is extended to streamline unit, integration, and system testing. First, we create the UML sequence diagrams, then we create both UML model and test cases (for unit, integration, and system testing) according to the sequence diagrams. Finally, we execute the test cases until the unit, integration, and system testing pass.

Results

We applied agile MDD to the development of a real-time telecommunication system successfully and released the product as planned. Our application system included five major components, two of which we modeled in UML for automatic code generation. Agile MDD applied to the development of these two UML-based autocoding components only. The development

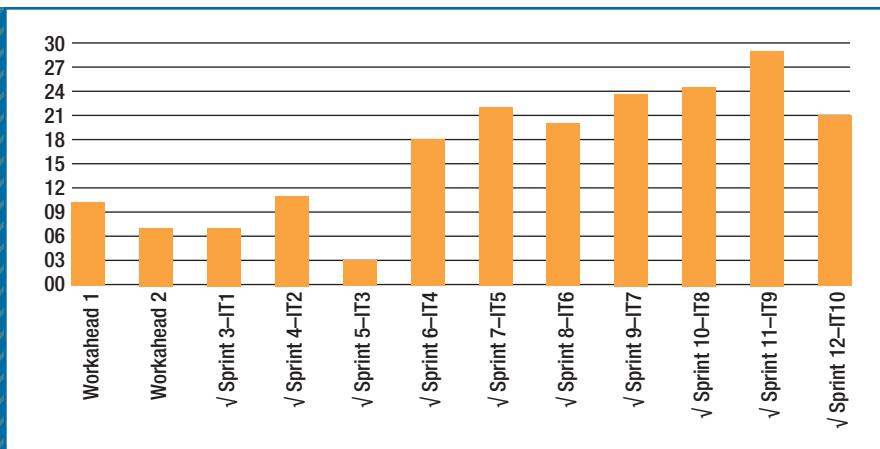


FIGURE 3. Sample sprint velocity chart in terms of number of story points per sprint. The development reached the lowest sprint velocity level in Sprint 5, and stabilized and gained traction starting at Sprint 6 and later.

ABOUT THE AUTHORS



YUEFENG ZHANG is a senior systems/application engineer at UniqueSoft, a company specializing in model-driven development and engineering. His research interests include model-driven development, agile software development processes, aspect and service-oriented modeling methodology and tools, and their applications to automatic code generation. Zhang has a PhD in computer science from the University of Western Ontario. Contact him at yuefeng.zhang@uniquesoft.com.



SHAILESH PATEL is a software engineering development manager at Motorola. His research interests include model-driven development, DFSS (Design for Six Sigma)-driven software development, agile software development processes, and software measurement. Patel has an MS in computer engineering from the University of Illinois. Contact him at CSG024@motorolasolutions.com.

of other hand-coded components followed the normal agile process. All the interfaces among these components are formally defined in formal languages—Interface Specification Language or Abstract Syntax Notation One (<http://www.obj-sys.com/asn1tutorial/asn1only.html>)—for later marshaling code generation.

As Figure 2 shows, an automatic code generation toolset maximizes automatic code generation. More than 93 percent of the entire component code was automatically generated. With such

a high percentage of code generation in conjunction with agile MDD practices, the productivity in number of source code lines per staff month increased threefold compared with hand coding. We also observed that the quality of automatically generated code in terms of defects density is significantly higher than manual code.

Regarding UML code generation, we observed that the more complicated the UML state machines, the higher the UML automatic code generation percentage. UML code generation alone is

limited in increasing the total code generation percentage. It's essential to automate data marshaling and platform integration code generation to significantly increase the total percentage of code generation.

We divided the entire software development life cycle into 10 iterations and 12 sprints. Early in the project, the team struggled to gain and maintain traction because we failed to follow agile MDD practices. We started the project as a brand-new team that formed in batches—not all team members joined at the same time. In particular, a significant portion of the team started at Sprint 6 in an overall plan of 12 sprints. Many team members had no MDD background or prior agile experience. In addition, most team members needed to pick up new domain knowledge.

As Figure 3 indicates, the development sprint velocity reached the lowest level in Sprint 5. To get the project back on track, we created a quick baseline of prior sprints because the agile MDD baseline wasn't available, established a revised plan before executing Sprint 6, eliminated manual detailed design documentation, and implemented test-driven modeling and other agile MDD practices. After taking these actions, the development stabilized and gained traction starting at Sprint 6 and going forward.

Similar to Mike Cohn's definition of velocity,⁸ the sprint velocity in Figure 3 is in terms of the number of story points (similar to use case scenarios) actually completed in each sprint. As we mentioned earlier, we used VersionOne for agile project planning and management. The data for Figure 3 comes from this tool. The story points are the units of work effort measured and tracked in VersionOne.

We observed that, as we approached the release due date, the defect closing rate couldn't catch up with the defect opening rate because certain types of testing (such as load or stress testing,

availability testing, and robust testing) happened in the last few weeks. We quickly got this under control by improving code inspection and the change request resolution process.

From the MDD perspective, the key to success is to maximize automation using the MDD tools chain to enable mistake-free (high-quality) development and significant productivity increase. From the agile perspective, the key is to efficiently achieve end-to-end iterations, from system engineering all the way down to system testing. This requires streamlining different process activities such as system engineering, development, and testing.

Agile MDD is still relatively new in real software development. The learning curve is sharp for any new organization to adopt due to process, culture, methodology, and other related changes. Thus, adopting a new agile MDD process is not likely to produce a short-term benefit. But, for the long term, it's ultimately worth it for large projects with multiple releases. 

Acknowledgments

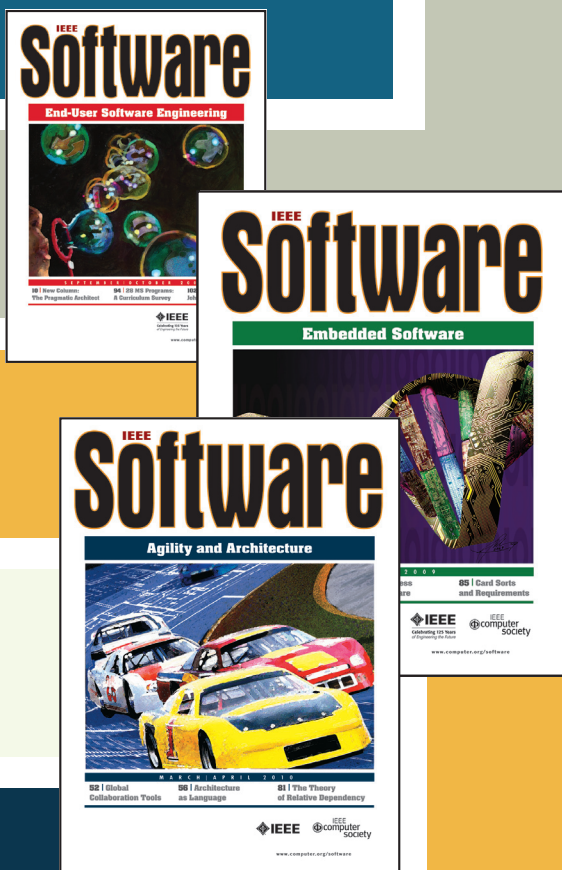
We thank the editors and reviewers for their constructive comments. We also thank Motorola managers (especially Cathal Tierney and Mike Wirtjes) for their strong support for agile model-driven development and approval of this article's publication.

References

1. K. Schwaber and M. Beedle, *Agile Software Development with SCRUM*, Pearson, 2002.
2. K. Beck, *Extreme Programming Explained*, Addison-Wesley, 2000.
3. I. Jacobson, G. Booch, and J. Rumbaugh, *The Unified Software Development Process*, Addison-Wesley, 1999.
4. T. Stahl and M. Volter, *Model-Driven Software Development*, John Wiley & Sons, 2006.
5. S.W. Ambler, *The Object Primer*, Cambridge Univ. Press, 2004.
6. Y. Zhang, "Test-Driven Modeling for Model Driven Development," *IEEE Software*, vol. 21, no. 5, 2004, pp. 80–86.
7. J. Ellsberger, D. Hogrefe, and A. Sarma, *SDL: Formal Object-Oriented Language for Communicating Systems*, Pearson, 1997.
8. M. Cohn, *Agile Estimating and Planning*, Pearson, 2006.



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.



**KEEP YOUR
COPY OF
IEEE SOFTWARE
FOR YOURSELF!**

Give subscriptions
to your colleagues
or as graduation or
promotion gifts—
way better than a tie!

IEEE Software
is the authority on
translating software
theory into practice.

www.computer.org/
software/**SUBSCRIBE**

SUBSCRIBE TODAY